LLD – Paper Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Paper module of OpenScholar. The module manages the lifecycle of research papers including draft creation, metadata management, versioning, multi-author handling, submission for moderation, retrieval, and related paper discovery.

The design is aligned with UI requirements and integrates with Storage, Author, Moderation, Search, Engagement, and Analytics modules.

---

2. Scope

The Paper module is responsible for:

- Create paper (draft)
- Update draft metadata
- Upload and manage PDF files
- Version control (multiple versions per paper)
- Manage authors per version (delegates to Author module)
- Submit paper for moderation
- Retrieve paper details (public + owner views)
- Retrieve related papers

Out of Scope:

- Moderation decisions (Moderation module)
- File storage internals (Storage module)
- Search indexing (Search module)

---

3. Data Model (Relevant Tables)

Tables used:

- papers
- paper_versions
- paper_authors
- authors
- paper_tags
- tags
- users

papers

- id (UUID)
- status (ENUM: draft | pending | approved | rejected)
- category_id (INT)
- created_by (UUID)
- is_deleted (BOOLEAN)
- comment_count (INT, default 0)
- reaction_count (INT, default 0)
- created_at (TIMESTAMP)

paper_versions

- id (UUID)
- paper_id (UUID)
- version_number (INT)
- title (TEXT)
- abstract (TEXT)
- keywords (TEXT[])
- pdf_url (TEXT)
- is_published (BOOLEAN)
- created_at (TIMESTAMP)

Constraints:

- UNIQUE (paper_id, version_number)
- Only one version per paper where is_published = true

---

4. API Contracts

4.1 Create Paper (Draft)

POST /api/papers

Request:

- multipart/form-data
- file (optional)
- metadata (optional)

Response: { "paperId": "uuid", "status": "draft" }

4.2 Update Draft

PUT /api/papers/{paperId}

Request:

- file (optional)
- metadata fields

4.3 Get Paper (Draft + Public)

GET /api/papers/{paperId}

Behavior:

- Owner → can see draft/pending
- Public → only approved

Response: { "id": "uuid", "status": "approved", "version": { "title": "...", "abstract": "...", "keywords": [], "pdfUrl": "..." }, "authors": [], "metrics": { "views": 0, "downloads": 0 } }

4.4 Submit Paper

POST /api/papers/{paperId}/submit

Validation:

- title, abstract, keywords required
- at least one author
- PDF required

4.5 Create New Version

POST /api/papers/{paperId}/versions

4.6 Get Related Papers

GET /api/papers/{paperId}/related

Response: { "results": [] }

---

5. Service Logic

5.1 createPaperDraft()

- Create paper (status = draft)
- Create version 1
- Upload file if provided

5.2 updateDraft()

- Validate ownership
- Update metadata
- Update version or replace file

5.3 submitPaper()

- Validate required fields
- Update status → pending

5.4 createNewVersion()

- Increment version number
- Upload new file
- Create new version
- Mark previous version unpublished

5.5 getPaper()

- Check access (owner/public)
- Fetch latest published version
- Join authors + metrics

5.6 getRelatedPapers()

- Match by:
- same category
- overlapping keywords
- Exclude current paper
- Limit results (5–10)

---

6. Validation Rules

- File must be PDF ≤ 50MB
- Required fields enforced on submit
- Author ordering validated (via Author module)

---

7. Security Considerations

- Only owner can edit draft
- Only approved papers visible publicly
- Validate JWT for protected routes

---

8. Performance Considerations

- Index on paper.status
- Index on paper_versions.paper_id
- Use aggregated metrics (paper_metrics)

---

9. Edge Cases

- Updating non-existent paper
- Submitting incomplete draft
- Concurrent version creation
- Missing latest version

---

10. Prisma Model Mapping

model Paper { id String @id @default(uuid()) status String categoryId Int? createdBy String isDeleted Boolean @default(false) commentCount Int @default(0) reactionCount Int @default(0) createdAt DateTime @default(now()) }

model PaperVersion { id String @id @default(uuid()) paperId String versionNumber Int title String abstract String keywords String[] pdfUrl String isPublished Boolean createdAt DateTime @default(now()) }

---

11. Folder Structure

src/ app/api/papers/route.ts app/api/papers/[id]/route.ts app/api/papers/[id]/submit/route.ts app/api/papers/[id]/versions/route.ts app/api/papers/[id]/related/route.ts

modules/paper/service.ts modules/paper/repository.ts

---

12. Acceptance Criteria

- User can create draft paper
- User can update draft

- User can submit paper
- Versioning works correctly
- Related papers returned correctly

---

13. Future Enhancements

- DOI integration
- Plagiarism check
- Version diff
- Auto-save drafts

End of Document